JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface Set<E>

Type Parameters:

`E - the type of elements maintained by this set`

All Superinterfaces:

`Collection<E>, Iterable<E>`

All Known Subinterfaces:

`NavigableSet<E>, SortedSet<E>`

All Known Implementing Classes:

`AbstractSet, ConcurrentHashMap.KeySetView, ConcurrentSkipListSet, CopyOnWriteArraySet, EnumSet, HashSet, JobStateReasons, LinkedHashSet, TreeSet`

```
public interface Set<E>
extends Collection<E>
```

A collection that contains no duplicate elements. More formally, sets contain no pair of elements `e1` and `e2` such that `e1.equals(e2)`, and at most one null element. As implied by its name, this interface models the mathematical *set* abstraction.

The `Set` interface places additional stipulations, beyond those inherited from the `Collection` interface, on the contracts of all constructors and on the contracts of the `add`, `equals` and `hashCode` methods. Declarations for other inherited methods are also included here for convenience. (The specifications accompanying these declarations have been tailored to the `Set` interface, but they do not contain any additional stipulations.)

The additional stipulation on constructors is, not surprisingly, that all constructors must create a set that contains no duplicate elements (as defined above).

Note: Great care must be exercised if mutable objects are used as set elements. The behavior of a set is not specified if the value of an object is changed in a manner that affects `equals` comparisons while the object is an element in the set. A special case of this prohibition is that it is not permissible for a set to contain itself as an element.

Some set implementations have restrictions on the elements that they may contain. For example, some implementations prohibit null elements, and some have restrictions on the types of their elements. Attempting to add an ineligible element throws an unchecked exception, typically `NullPointerException` or `ClassCastException`. Attempting to query the presence of an ineligible element may throw an exception, or it may simply return false; some implementations will exhibit the former behavior and some will exhibit the latter. More generally, attempting an operation on an ineligible element whose completion would not result in the insertion of an ineligible element into the set may throw an exception or it may succeed, at the option of the implementation. Such exceptions are marked as "optional" in the specification for this interface.

This interface is a member of the Java Collections Framework.

Since:

1.2

See Also:

`Collection`, `List`, `SortedSet`, `HashSet`, `TreeSet`, `AbstractSet`, `Collections.singleton(java.lang.Object)`, `Collections.EMPTY_SET`

Method Summary

Abstract Methods **Default Methods**

Modifier and Type	Method	Description
boolean	**add**(**E** e)	Adds the specified element to this set if it is not already present (optional operation).
boolean	**addAll**(**Collection**<? extends **E**> c)	Adds all of the elements in the specified collection to this set if they're not already present (optional operation).

void	**clear**()	Removes all of the elements from this set (optional operation).
boolean	**contains**(**Object** o)	Returns `true` if this set contains the specified element.
boolean	**containsAll**(**Collection**<?> c)	Returns `true` if this set contains all of the elements of the specified collection.
boolean	**equals**(**Object** o)	Compares the specified object with this set for equality.
int	**hashCode**()	Returns the hash code value for this set.
boolean	**isEmpty**()	Returns `true` if this set contains no elements.
Iterator<**E**>	**iterator**()	Returns an iterator over the elements in this set.
boolean	**remove**(**Object** o)	Removes the specified element from this set if it is present (optional operation).
boolean	**removeAll**(**Collection**<?> c)	Removes from this set all of its elements that are contained in the specified collection (optional operation).

boolean	**retainAll**(**Collection**<? > c)	Retains only the elements in this set that are contained in the specified collection (optional operation).
int	**size**()	Returns the number of elements in this set (its cardinality).
default **Spliterator**<**E**>	**spliterator**()	Creates a `Spliterator` over the elements in this set.
Object[]	**toArray**()	Returns an array containing all of the elements in this set.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this set; the runtime type of the returned array is that of the specified array.

Methods inherited from interface java.util.Collection

`parallelStream`, `removeIf`, `stream`

Methods inherited from interface java.lang.Iterable

`forEach`

Method Detail

size

`int size()`

Returns the number of elements in this set (its cardinality). If this set contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Specified by:

`size` in interface `Collection<E>`

Returns:

the number of elements in this set (its cardinality)

isEmpty

`boolean isEmpty()`

Returns `true` if this set contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Returns:

true if this set contains no elements

contains

`boolean contains(Object o)`

Returns `true` if this set contains the specified element. More formally, returns `true` if and only if this set contains an element `e` such that `(o==null ? e==null : o.equals(e))`.

Specified by:

`contains` in interface `Collection<E>`

Parameters:

o - element whose presence in this set is to be tested

Returns:

true if this set contains the specified element

Throws:

`ClassCastException` - if the type of the specified element is incompatible with this set (`optional`)

NullPointerException - if the specified element is null and this set does not permit null elements (optional)

iterator

Iterator<E> iterator()

Returns an iterator over the elements in this set. The elements are returned in no particular order (unless this set is an instance of some class that provides a guarantee).

Specified by:

iterator in interface Collection<E>

Specified by:

iterator in interface Iterable<E>

Returns:

an iterator over the elements in this set

toArray

Object[] toArray()

Returns an array containing all of the elements in this set. If this set makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

The returned array will be "safe" in that no references to it are maintained by this set. (In other words, this method must allocate a new array even if this set is backed by an array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

toArray in interface Collection<E>

Returns:

an array containing all the elements in this set

toArray

<T> T[] toArray(T[] a)

Returns an array containing all of the elements in this set; the runtime type of the returned array is that of the specified array. If the set fits in the

specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this set.

If this set fits in the specified array with room to spare (i.e., the array has more elements than this set), the element in the array immediately following the end of the set is set to `null`. (This is useful in determining the length of this set *only* if the caller knows that this set does not contain any null elements.)

If this set makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a set known to contain only strings. The following code can be used to dump the set into a newly allocated array of `String`:

```
String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Type Parameters:

`T - the runtime type of the array to contain the collection`

Parameters:

`a - the array into which the elements of this set are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose.`

Returns:

`an array containing all the elements in this set`

Throws:

`ArrayStoreException - if the runtime type of the specified array is not a supertype of the runtime type of every element in this set`

`NullPointerException - if the specified array is null`

add

`boolean add(E e)`

Adds the specified element to this set if it is not already present (optional operation). More formally, adds the specified element `e` to this set if the set contains no element `e2` such that `(e==null ? e2==null : e.equals(e2))`.

If this set already contains the element, the call leaves the set unchanged and returns `false`. In combination with the restriction on constructors, this ensures that sets never contain duplicate elements.

The stipulation above does not imply that sets must accept all elements; sets may refuse to add any particular element, including `null`, and throw an exception, as described in the specification for `Collection.add`. Individual set implementations should clearly document any restrictions on the elements that they may contain.

Specified by:

`add` in interface `Collection<E>`

Parameters:

`e - element to be added to this set`

Returns:

`true if this set did not already contain the specified element`

Throws:

`UnsupportedOperationException - if the add operation is not supported by this set`

`ClassCastException - if the class of the specified element prevents it from being added to this set`

`NullPointerException - if the specified element is null and this set does not permit null elements`

`IllegalArgumentException - if some property of the specified element prevents it from being added to this set`

remove

`boolean remove(Object o)`

Removes the specified element from this set if it is present (optional operation). More formally, removes an element `e` such that `(o==null ? e==null : o.equals(e))`, if this set contains such an element. Returns `true` if this set contained the element (or equivalently, if this set changed as a result of the call). (This set will not contain the element once the call returns.)

Specified by:

`remove` in interface `Collection<E>`

Parameters:

`o - object to be removed from this set, if present`

Returns:

`true if this set contained the specified element`

Throws:

ClassCastException - if the type of the specified element is incompatible with this set (optional)

NullPointerException - if the specified element is null and this set does not permit null elements (optional)

UnsupportedOperationException - if the remove operation is not supported by this set

containsAll

boolean containsAll(Collection<?> c)

Returns true if this set contains all of the elements of the specified collection. If the specified collection is also a set, this method returns true if it is a *subset* of this set.

Specified by:

containsAll in interface Collection<E>

Parameters:

c - collection to be checked for containment in this set

Returns:

true if this set contains all of the elements of the specified collection

Throws:

ClassCastException - if the types of one or more elements in the specified collection are incompatible with this set (optional)

NullPointerException - if the specified collection contains one or more null elements and this set does not permit null elements (optional), or if the specified collection is null

See Also:

contains(Object)

addAll

boolean addAll(Collection<? extends E> c)

Adds all of the elements in the specified collection to this set if they're not already present (optional operation). If the specified collection is also a set, the addAll operation effectively modifies this set so that its value is the *union* of the two sets. The behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

addAll in interface Collection<E>

Parameters:

c - collection containing elements to be added to this set

Returns:

true if this set changed as a result of the call

Throws:

UnsupportedOperationException - if the addAll operation is not supported by this set

ClassCastException - if the class of an element of the specified collection prevents it from being added to this set

NullPointerException - if the specified collection contains one or more null elements and this set does not permit null elements, or if the specified collection is null

IllegalArgumentException - if some property of an element of the specified collection prevents it from being added to this set

See Also:

add(Object)

retainAll

boolean retainAll(Collection<?> c)

Retains only the elements in this set that are contained in the specified collection (optional operation). In other words, removes from this set all of its elements that are not contained in the specified collection. If the specified collection is also a set, this operation effectively modifies this set so that its value is the *intersection* of the two sets.

Specified by:

retainAll in interface Collection<E>

Parameters:

c - collection containing elements to be retained in this set

Returns:

true if this set changed as a result of the call

Throws:

UnsupportedOperationException - if the retainAll operation is not supported by this set

ClassCastException - if the class of an element of this set is incompatible with the specified collection (optional)

NullPointerException - if this set contains a null element and the specified collection does not permit null elements (optional), or if the specified collection is null

See Also:

remove(Object)

removeAll

boolean removeAll(Collection<?> c)

Removes from this set all of its elements that are contained in the specified collection (optional operation). If the specified collection is also a set, this operation effectively modifies this set so that its value is the *asymmetric set difference* of the two sets.

Specified by:

removeAll in interface Collection<E>

Parameters:

c - collection containing elements to be removed from this set

Returns:

true if this set changed as a result of the call

Throws:

UnsupportedOperationException - if the removeAll operation is not supported by this set

ClassCastException - if the class of an element of this set is incompatible with the specified collection (optional)

NullPointerException - if this set contains a null element and the specified collection does not permit null elements (optional), or if the specified collection is null

See Also:

remove(Object), contains(Object)

clear

void clear()

Removes all of the elements from this set (optional operation). The set will be empty after this call returns.

Specified by:

clear in interface Collection<E>

Throws:

UnsupportedOperationException - if the clear method is not supported by this set

equals

`boolean equals(Object o)`

Compares the specified object with this set for equality. Returns `true` if the specified object is also a set, the two sets have the same size, and every member of the specified set is contained in this set (or equivalently, every member of this set is contained in the specified set). This definition ensures that the equals method works properly across different implementations of the set interface.

Specified by:

`equals` in interface `Collection<E>`

Overrides:

`equals` in class `Object`

Parameters:

`o - object to be compared for equality with this set`

Returns:

`true if the specified object is equal to this set`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

`int hashCode()`

Returns the hash code value for this set. The hash code of a set is defined to be the sum of the hash codes of the elements in the set, where the hash code of a `null` element is defined to be zero. This ensures that `s1.equals(s2)` implies that `s1.hashCode()==s2.hashCode()` for any two sets `s1` and `s2`, as required by the general contract of `Object.hashCode()`.

Specified by:

`hashCode` in interface `Collection<E>`

Overrides:

`hashCode` in class `Object`

Returns:

`the hash code value for this set`

See Also:

`Object.equals(Object)`, `equals(Object)`

spliterator

`default Spliterator<E> spliterator()`

Creates a `Spliterator` over the elements in this set.

The `Spliterator` reports `Spliterator.DISTINCT`. Implementations should document the reporting of additional characteristic values.

Specified by:

`spliterator` in interface `Collection<E>`

Specified by:

`spliterator` in interface `Iterable<E>`

Implementation Requirements:

The default implementation creates a *late-binding* spliterator from the set's Iterator. The spliterator inherits the *fail-fast* properties of the set's iterator.

The created Spliterator additionally reports `Spliterator.SIZED`.

Implementation Note:

The created Spliterator additionally reports `Spliterator.SUBSIZED`.

Returns:

a Spliterator over the elements in this set

Since:

1.8

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.